

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет безопасности информационных технологий

Дисциплина:

«Вычислительные сети и контроль безопасности в компьютерных сетях»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №3

«Настройка межсетевого экрана»

Выполнили:

Бардышев Артём Антонович,
студент группы N3346



(подпись)

Суханкулиев Мухаммет,
студент группы N3346



(подпись)

Шангина Елизавета Кирилловна,
студентка группы N3346



(подпись)

Проверил:

Савков Сергей Витальевич,
инженер, ФБИТ

(отметка о выполнении)

(подпись)

Санкт-Петербург

2026 г.

СОДЕРЖАНИЕ

Введение	3
1 Подготовка среды моделирования	4
1.1 Создание виртуальных машин	4
1.2 Настройка сетевых интерфейсов.....	7
1.2.1 Настройка IPtablesFW,	7
1.2.2 Настройка ExternalClient.....	7
1.2.3 Настройка InternalClient	7
1.3 Базовая связность	8
1.3.1 IPtablesFW	8
1.3.2 ExternalClient.....	8
1.3.3 На InternalClient	8
2 Настройка iptables	10
2.1 Включение IP forwarding	10
2.2 Применение правил iptables.....	10
2.2.1 Блокировка больших пакетов.....	11
2.2.2 Защита от brute force.....	11
2.2.3 Разрешить уже установленные соединения.....	12
2.2.4 Исходящий трафик - только SSH и веб	12
3 Тестирование.....	14
3.1 Проверка исходящего трафика.....	14
3.2 Проверка блокировки больших пакетов с TTL > 10	14
3.3 NAT	15
3.4 Защита от brute force.....	16
Заключение.....	18
Список использованных источников.....	20

ВВЕДЕНИЕ

Цель работы – изучить на примере netfilter/iptables основные принципы работы межсетевых экранов. Освоить базовую настройку правил iptables.

Для достижения поставленной цели необходимо решить следующие задачи:

1. настроить лабораторный стенд, включающий шлюз/межсетевой экран, внешний клиент и внутренний клиент;
2. настроить маршрутизацию проходящего трафика на локальном сервере;
3. выполнить необходимые настройки межсетевого экрана на локальном сервере в соответствии с заданием;
4. протестировать работу выполненных настроек;

1 ПОДГОТОВКА СРЕДЫ МОДЕЛИРОВАНИЯ

1.1 Создание виртуальных машин

Файл → Создать

- Имя: `Kali-IPtablesFW`
- Тип: Linux
- Версия: Debian (64-bit)
- ОЗУ: 2048 MB (2 GB)
- Жёсткий диск: Создать виртуальный жёсткий диск сейчас
- Размер: 20 GB, VDI, Динамически выделяемый

Далее мы клонируем эту виртуальную машину два раза

Для клонов – меняем имя и настройки сети

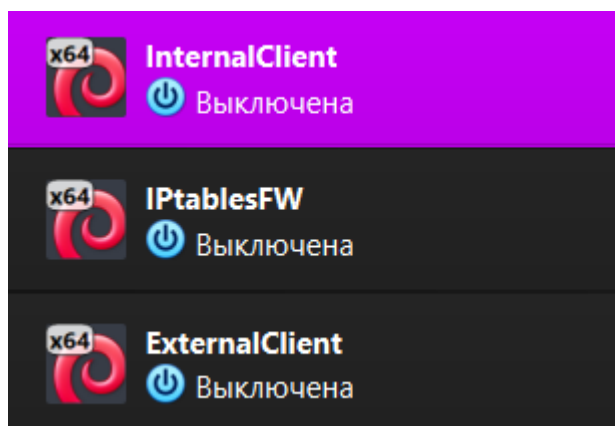


Рисунок 1 – используемые виртуальные машины

Обозначения VM:

- IPtablesFW = Шлюз/Межсетевой экран
- ExternalClient = Внешний клиент
- InternalClient = Внутренний клиент

Для IPtablesFW:

1. Выберите VM → Настройки → Сеть
2. Адаптер 1 (Внешний):
 - Включить сетевой адаптер
 - Подключение к: "Внутренняя сеть"
 - Имя: `external-net`
 - Кабель подключен

3. Адаптер 2 (Внутренний):

- Включить сетевой адаптер
- Подключение к: "Внутренняя сеть"
- Имя: `internal-net`
- Кабель подключен

4. Адаптер 3 и Адаптер 4: - Отключить

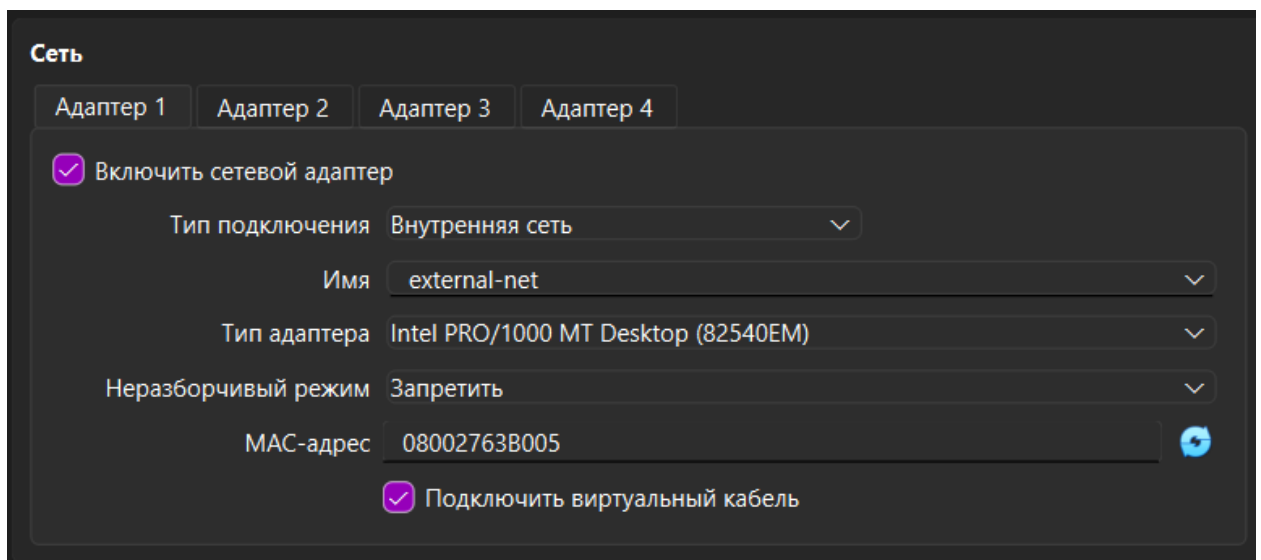


Рисунок 2 – Настройка сетевого адаптера 1 для IPtablesFW

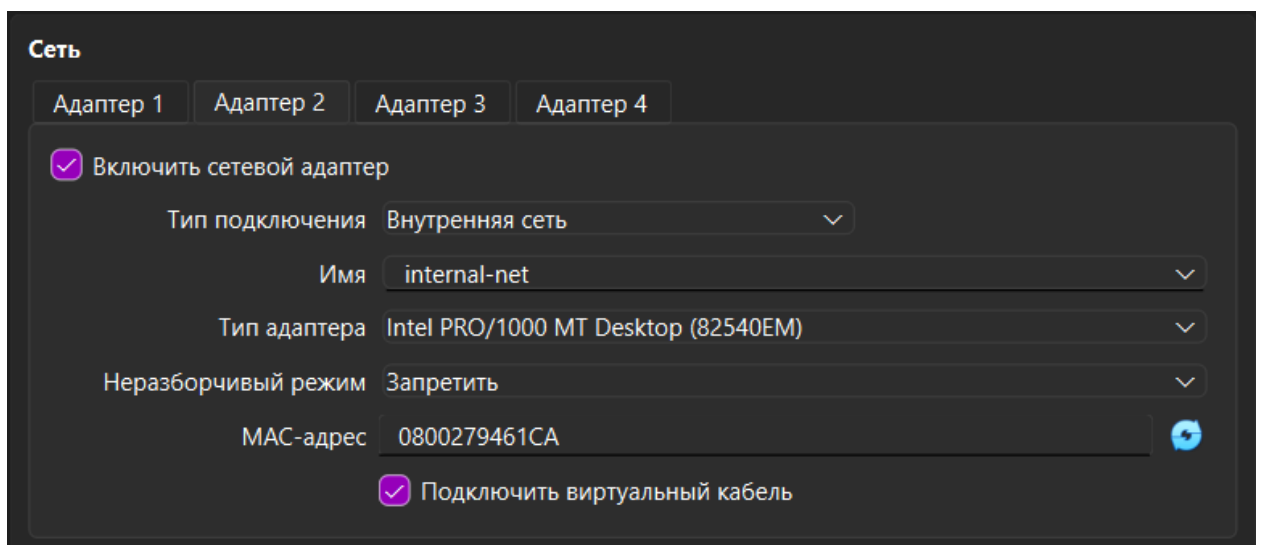


Рисунок 3 - Настройка сетевого адаптера 2 для IPtablesFW

Для ExternalClient:

1. Настройки → Сеть

2. Адаптер 1:

- Включить сетевой адаптер
- Подключение к: "Внутренняя сеть"
- Имя: `external-net` (то же имя, что у IPtablesFW адаптера 1!)

- Кабель подключен

3. Адаптер 2, 3, 4: - Отключить

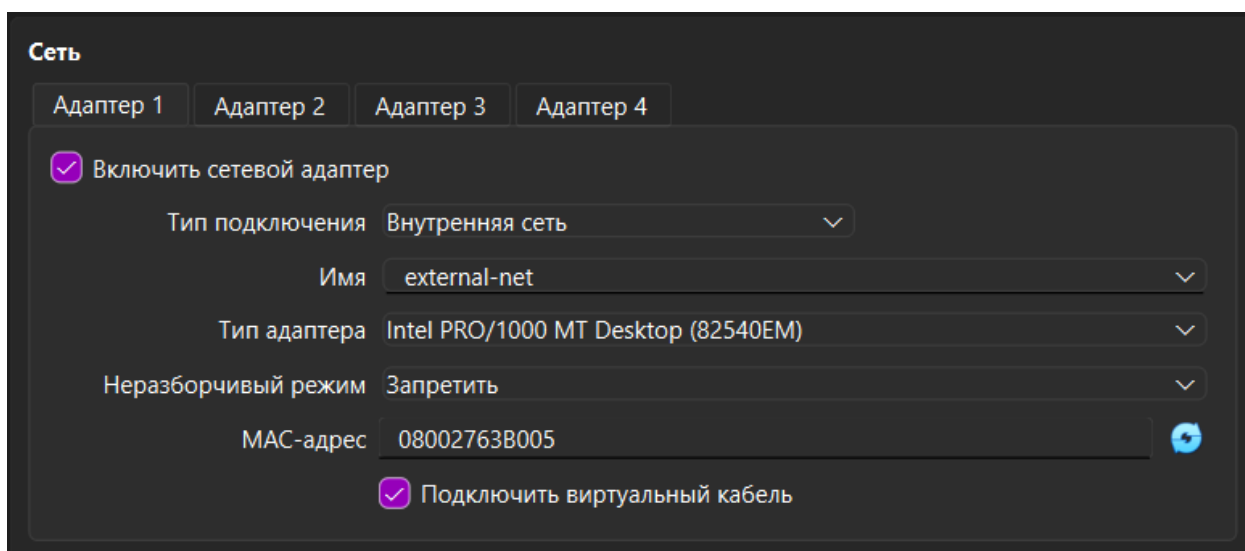


Рисунок 4 - Настройки сети для ExternalClient

Для InternalClient:

1. Настройки → Сеть

2. Адаптер 1:

- Включить сетевой адаптер

- Подключение к: "Внутренняя сеть"

- Имя: `internal-net` (то же имя, что у IPtablesFW адаптера 2!)

- Кабель подключен

3. Адаптер 2, 3, 4: - Отключить

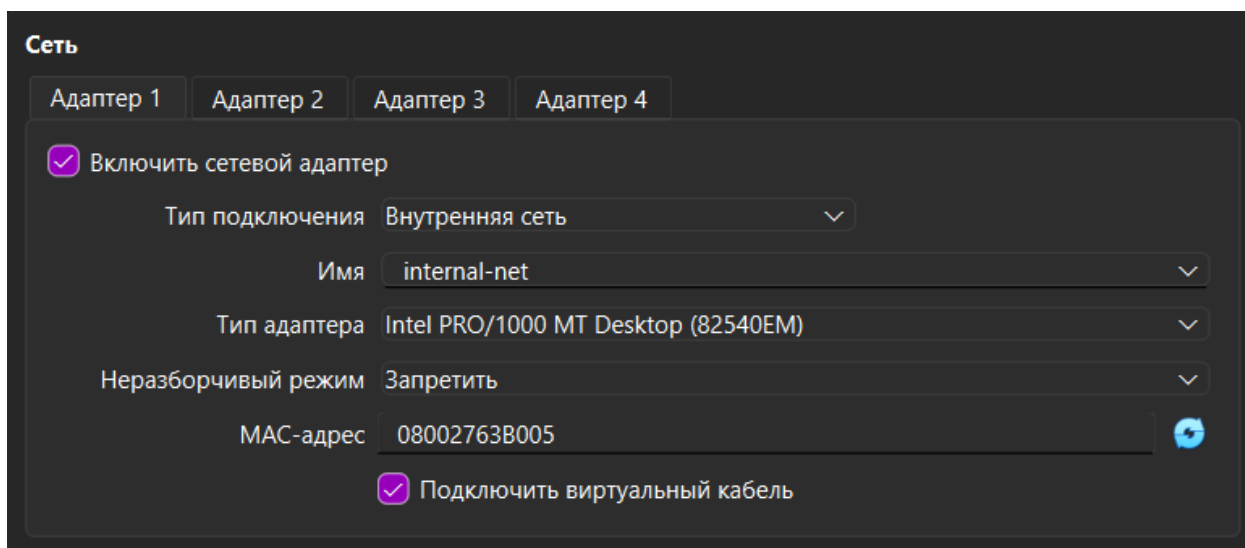


Рисунок 5 – Настройки сети для InternalClient

1.2 Настройка сетевых интерфейсов

1.2.1 Настройка IPtablesFW,

Отключаем NetworkManager для интерфейсов, настраиваем внешний интерфейс, настраиваем внутренний интерфейс и включаем IP forwarding

```
sudo su
nmcli device set eth0 managed no 2>/dev/null
nmcli device set eth1 managed no 2>/dev/null
ip addr add 192.168.1.1/24 dev eth0
ip link set eth0 up
ip addr add 10.0.0.1/24 dev eth1
ip link set eth1 up
echo 1 > /proc/sys/net/ipv4/ip_forward
ip addr show
```

NetworkManager: Это служба управления сетью в Linux, которая автоматически настраивает сетевые интерфейсы. Отключение управления (`managed no`) позволяет вручную настраивать интерфейсы без конфликтов.

IP-адрес с маской /24: Запись `192.168.1.1/24` означает IP-адрес 192.168.1.1 с маской подсети 255.255.255.0 (24 бита для сети). Это означает, что в сети может быть до 254 хостов (192.168.1.1-192.168.1.254).

IP forwarding: Это параметр ядра Linux, который позволяет системе работать как маршрутизатор, пересылая пакеты между сетевыми интерфейсами. Без включенного IP forwarding пакеты не будут пересылаться между интерфейсами, даже если правила iptables разрешают. Значение 1 включает forwarding, 0 - отключает.

1.2.2 Настройка ExternalClient

```
sudo su
nmcli device set eth0 managed no 2>/dev/null
ip addr add 192.168.1.10/24 dev eth0
ip link set eth0 up
ip route add default via 192.168.1.1
ip addr show eth0
ip route show
```

Маршрут по умолчанию (`default via`) указывает системе, куда отправлять пакеты, для которых нет явного маршрута в таблице маршрутизации. Это шлюз по умолчанию (gateway), который обычно является маршрутизатором, соединяющим локальную сеть с другими сетями. Команда `ping` использует протокол ICMP для проверки доступности хоста - она отправляет ICMP echo request и ожидает ICMP echo reply.

1.2.3 Настройка InternalClient

```
sudo su
```

```
nmcli device set eth0 managed no 2>/dev/null
ip addr add 10.0.0.10/24 dev eth0
ip link set eth0 up
ip route add default via 10.0.0.1
ip addr show eth0
ip route show
```

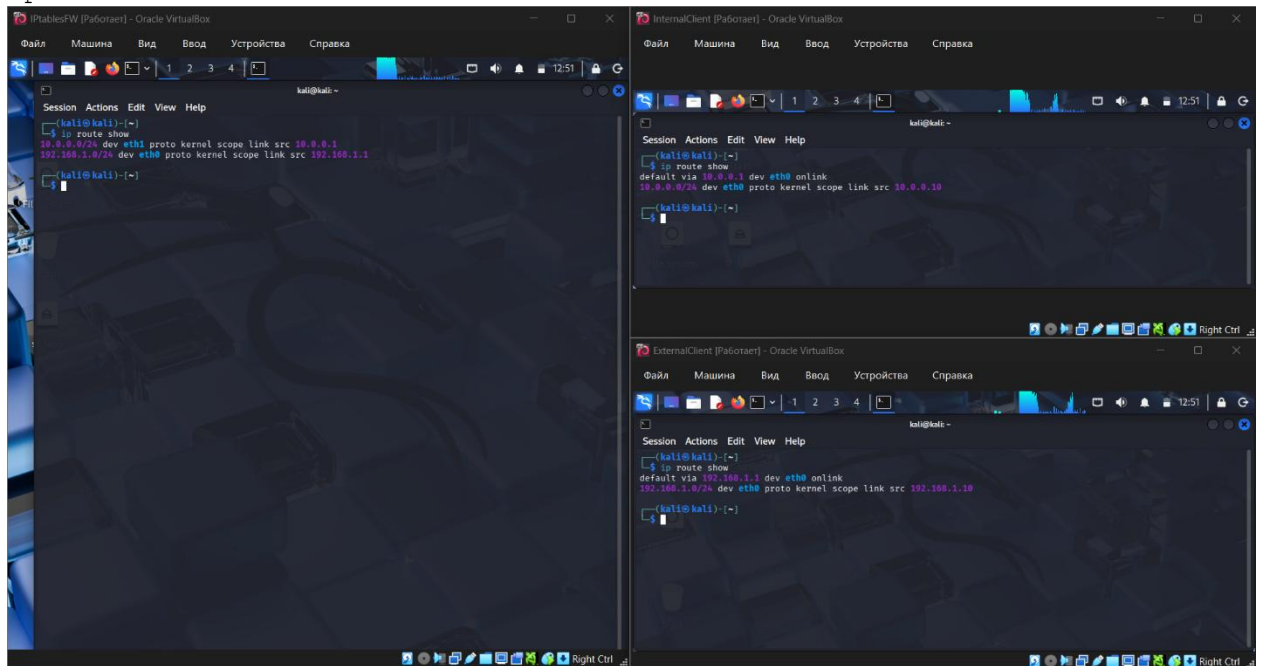


Рисунок 6 – Настройка виртуальных машин

1.3 Базовая связность

1.3.1 IPtablesFW

Проверка связи с внешней сетью

```
ping -c 3 192.168.1.10
```

Проверка связи с внутренней сетью

```
ping -c 3 10.0.0.10
```

1.3.2 ExternalClient

Проверка связи с шлюзом

```
ping -c 3 192.168.1.1
```

1.3.3 На InternalClient

Проверка связи с шлюзом


```
ping -c 3 10.0.0.1
```

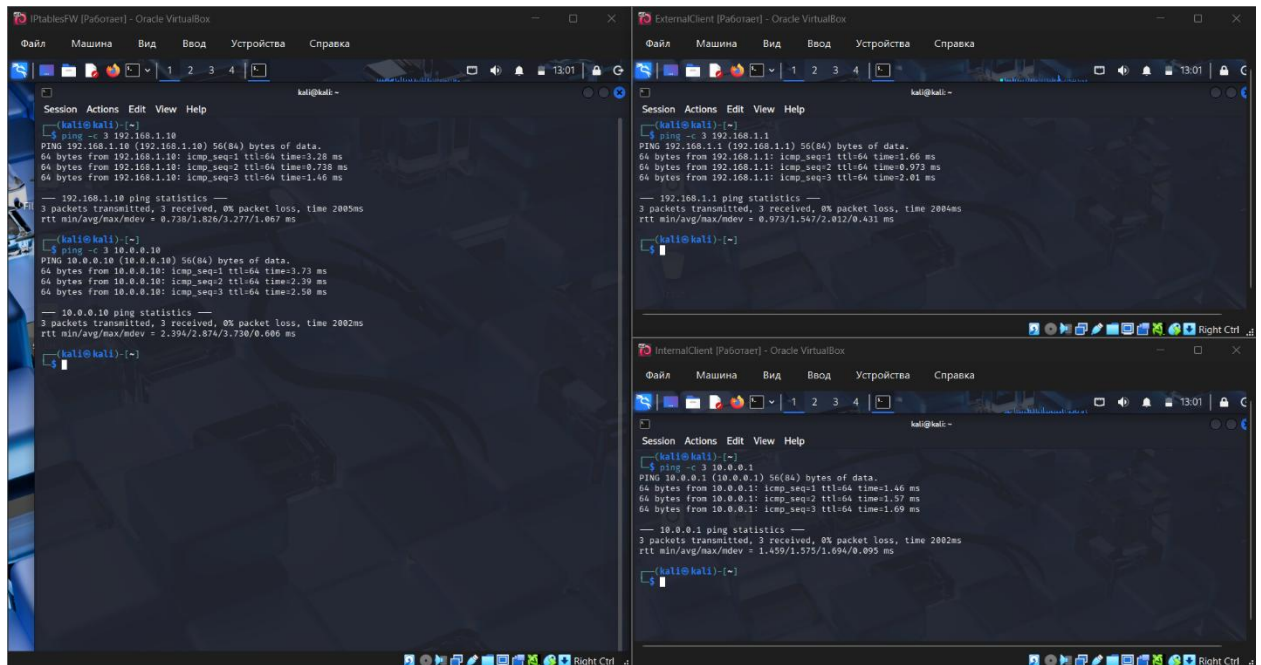


Рисунок 7 – проверка связи между виртуальными машинами

2 НАСТРОЙКА IPTABLES

2.1 Включение IP forwarding

IP forwarding (пересылка IP-пакетов) - это функция ядра Linux, которая позволяет системе работать как маршрутизатор. Когда forwarding включен, ядро может пересылать пакеты между сетевыми интерфейсами. Без этого даже при наличии правил iptables FORWARD пакеты не будут пересылаться. `/proc/sys/net/ipv4/ip\_forward` - это файл в виртуальной файловой системе proc, который напрямую связан с параметром ядра. Запись в `/etc/sysctl.conf` делает настройку постоянной - она применяется при каждой загрузке системы.

В консоли каждой виртуальной машины (далее VM) включаем IP forwarding вручную

```
echo 1 > /proc/sys/net/ipv4/ip_forward
echo "net.ipv4.ip_forward=1" >> /etc/sysctl.conf
sysctl -p
cat /proc/sys/net/ipv4/ip_forward
```

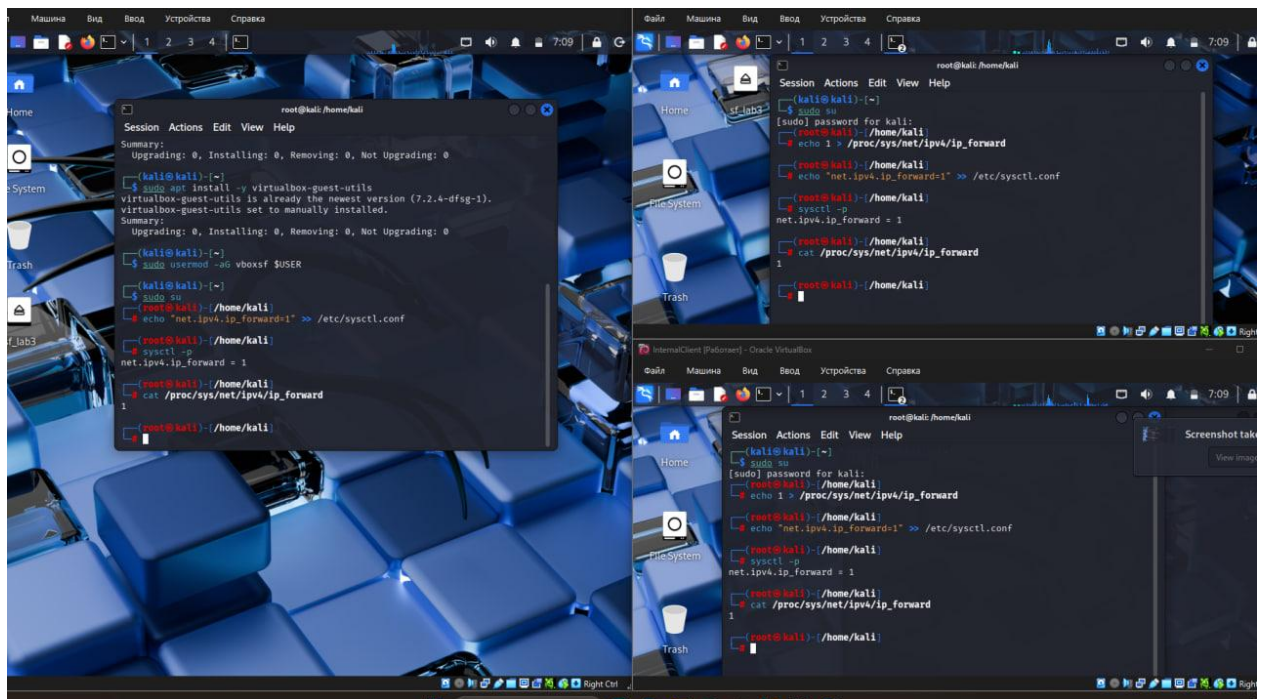


Рисунок 8 – IP forwarding включен на всех VM

2.2 Применение правил iptables

Правила применяются в определенном порядке. Сначала настраиваем блокировки, затем разрешения.

- `iptables -F`: Очищает (flush) все правила из цепочек. `-F` без параметров очищает все цепочки в таблице filter.

- `iptables -X`: Удаляет пользовательские цепочки (custom chains), которые были созданы командой `-N`.

- `-t nat -F` и `-t nat -X`: Аналогично для таблицы NAT.

- Политики по умолчанию (`-P`): Это действие, которое применяется к пакетам, не соответствующим ни одному правилу в цепочке. В начале настройки устанавливаем АССЕРТ для безопасности - если что-то пойдет не так, трафик не будет заблокирован. После настройки всех правил можно изменить политики на DROP для более строгой защиты.

Очистка существующих правил

```
iptables -F
iptables -X
iptables -t nat -F
iptables -t nat -X
```

Политики по умолчанию (АССЕРТ для начала)

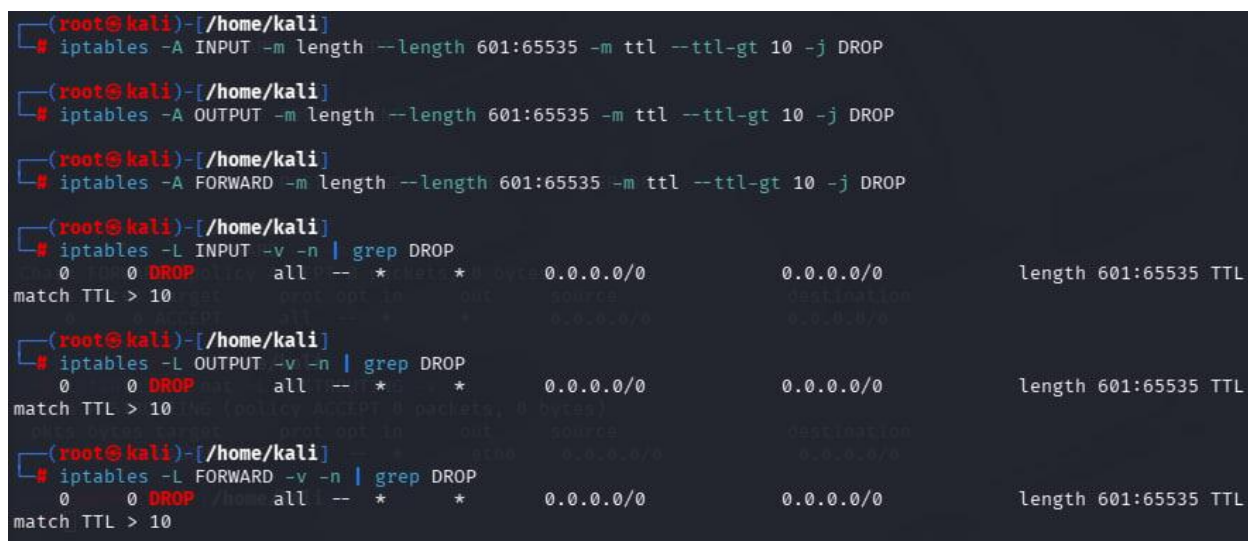
```
iptables -P INPUT ACCEPT
iptables -P FORWARD ACCEPT
iptables -P OUTPUT ACCEPT
```

2.2.1 Блокировка больших пакетов

Блокируем пакеты размером > 600 байт с TTL > 10

Применяем к входящему, исходящему и транзитному трафику

```
iptables -A INPUT -m length --length 601:65535 -m ttl --ttl-gt 10 -j DROP
iptables -A OUTPUT -m length --length 601:65535 -m ttl --ttl-gt 10 -j DROP
iptables -A FORWARD -m length --length 601:65535 -m ttl --ttl-gt 10 -j DROP
```



```
(root@kali)-[/home/kali]
# iptables -A INPUT -m length --length 601:65535 -m ttl --ttl-gt 10 -j DROP

(root@kali)-[/home/kali]
# iptables -A OUTPUT -m length --length 601:65535 -m ttl --ttl-gt 10 -j DROP

(root@kali)-[/home/kali]
# iptables -A FORWARD -m length --length 601:65535 -m ttl --ttl-gt 10 -j DROP

(root@kali)-[/home/kali]
# iptables -L INPUT -v -n | grep DROP
0 0 DROP all -- * * 0.0.0.0/0 0.0.0.0/0 length 601:65535 TTL
match TTL > 10

(root@kali)-[/home/kali]
# iptables -L OUTPUT -v -n | grep DROP
0 0 DROP all -- * * 0.0.0.0/0 0.0.0.0/0 length 601:65535 TTL
match TTL > 10

(root@kali)-[/home/kali]
# iptables -L FORWARD -v -n | grep DROP
0 0 DROP all -- * * 0.0.0.0/0 0.0.0.0/0 length 601:65535 TTL
match TTL > 10
```

Рисунок 9 – Пакеты с TTL > 10 блокируются

2.2.2 Защита от brute force

Создаем отдельную цепочку для защиты SSH

```

iptables -N SSH_BRUTEFORCE
# Добавляем адрес в список при попытке SSH подключения
iptables -A INPUT -p tcp --dport 22 -m state --state NEW -j SSH_BRUTEFORCE
iptables -A SSH_BRUTEFORCE -m recent --set --name SSH -rsource
iptables -A SSH_BRUTEFORCE -m recent --update --seconds 600 --hitcount 5 --
name SSH --rsource -j DROP
# Если за 600 секунд (10 минут) было 5+ попыток - блокируем
# Если не заблокирован - разрешаем SSH
iptables -A SSH_BRUTEFORCE -j ACCEPT
# Применяем цепочку к входящему SSH трафику (порт 22)
iptables -A INPUT -p tcp --dport 22 -j SSH_BRUTEFORCE

```

```

(root@kali)-[/home/kali]
# iptables -N SSH_BRUTEFORCE

(root@kali)-[/home/kali]
# iptables -A SSH_BRUTEFORCE -m recent --set --name SSH --rsource

(root@kali)-[/home/kali]
# iptables -A SSH_BRUTEFORCE -m recent --update --seconds 600 --hitcount 5 --name SSH --rsource -j DROP

(root@kali)-[/home/kali]
# iptables -A SSH_BRUTEFORCE -j ACCEPT

(root@kali)-[/home/kali]
# iptables -A INPUT -p tcp --dport 22 -j SSH_BRUTEFORCE

(root@kali)-[/home/kali]
# iptables -L SSH_BRUTEFORCE -v -n
Chain SSH_BRUTEFORCE (1 references)
pkts bytes target prot opt in out source destination
0 0 DROP all -- * * 0.0.0.0/0 0.0.0.0/0 recent: SET name: SSH
side: source mask: 255.255.255.255
0 0 DROP all -- * * 0.0.0.0/0 0.0.0.0/0 recent: UPDATE second
s: 600 hit_count: 5 name: SSH side: source mask: 255.255.255.255
0 0 ACCEPT all -- * * 0.0.0.0/0 0.0.0.0/0

(root@kali)-[/home/kali]
# iptables -L INPUT -v -n | grep SSH
Chain INPUT (policy MASQUERADE)
0 0 SSH_BRUTEFORCE tcp -- * * 0.0.0.0/0 0.0.0.0/0 tcp dpt:22

```

Рисунок 10 – больше пяти попыток подключения по SSH блокируется

2.2.3 Разрешить уже установленные соединения

Важно: Это должно быть перед блокировкой OUTPUT

Позволяет работать уже установленным соединениям

```

iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
iptables -A OUTPUT -m state --state ESTABLISHED,RELATED -j ACCEPT

```

2.2.4 Исходящий трафик - только SSH и веб

Разрешаем loopback интерфейс

```

iptables -A OUTPUT -o lo -j ACCEPT

```

Разрешаем исходящий трафик на порты SSH (22), HTTP (80), HTTPS (443)

```

iptables -A OUTPUT -p tcp --dport 22 -j ACCEPT
iptables -A OUTPUT -p tcp --dport 80 -j ACCEPT
iptables -A OUTPUT -p tcp --dport 443 -j ACCEPT

```

Блокируем весь остальной исходящий трафик

```
iptables -A OUTPUT -j DROP
```

```
(root@kali)-[/home/kali]
# iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
# iptables -A OUTPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
# iptables -L INPUT -v -n | grep ESTABLISHED
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source            destination
0      0      0 ACCEPT    all  --  *      *        0.0.0.0/0          0.0.0.0/0          state RELATED,ESTABLISHED
# iptables -L OUTPUT -v -n | grep ESTABLISHED
Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target     prot opt out    out    source            destination
0      0      0 ACCEPT    all  --  *      *        0.0.0.0/0          0.0.0.0/0          state RELATED,ESTABLISHED
```

Рисунок 11 – Разрешение уже установленных соединений

```
(root@kali)-[/home/kali]
# iptables -A OUTPUT -o lo -j ACCEPT
# iptables -A OUTPUT -p tcp --dport 22 -j ACCEPT
# iptables -A OUTPUT -p tcp --dport 80 -j ACCEPT
# iptables -A OUTPUT -p tcp --dport 443 -j ACCEPT
# iptables -A OUTPUT -j DROP
# iptables -L OUTPUT -v -n --line-numbers
Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source            destination
1      0      0 DROP      all  --  *      *        0.0.0.0/0          0.0.0.0/0          length 601:65535
2      0      0 ACCEPT    all  --  *      *        0.0.0.0/0          0.0.0.0/0          state RELATED,ESTABLISHED
3      0      0 ACCEPT    all  --  *      lo       0.0.0.0/0          0.0.0.0/0
4      0      0 ACCEPT    tcp  --  *      *        0.0.0.0/0          0.0.0.0/0          tcp dpt:22
5      0      0 ACCEPT    tcp  --  *      *        0.0.0.0/0          0.0.0.0/0          tcp dpt:80
6      0      0 ACCEPT    tcp  --  *      *        0.0.0.0/0          0.0.0.0/0          tcp dpt:443
7      0      0 DROP      all  --  *      *        0.0.0.0/0          0.0.0.0/0
```

Рисунок 12 - Блокировка исходящего трафика (кроме SSH и веб)

3 ТЕСТИРОВАНИЕ

3.1 Проверка исходящего трафика

Попробуем подключиться к заблокированному порту (например, SMTP 25)

```
telnet 192.168.1.10 25
```

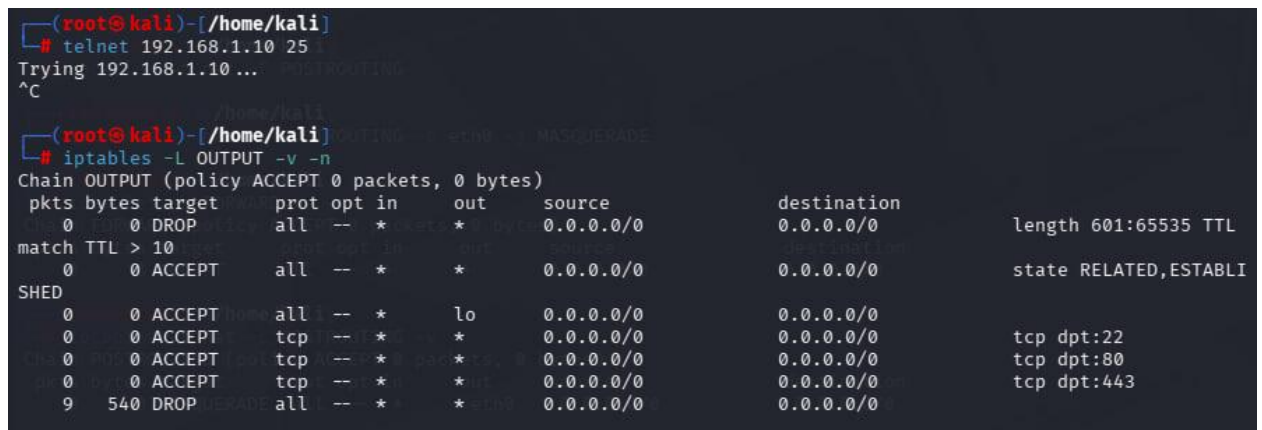
Должно быть заблокировано правилом OUTPUT DROP

Соединение не установится или будет таймаут

Проверка счетчиков правил

```
iptables -L OUTPUT -v -n
```

В колонке "pkts" должны увеличиваться счетчики для правил DROP



```
(root@kali)-[/home/kali]
# telnet 192.168.1.10 25
Trying 192.168.1.10 ...
^C

(root@kali)-[/home/kali]
# iptables -L OUTPUT -v -n
Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target prot opt in out source destination length 601:65535 TTL
0 0 DROP all -- * * 0.0.0.0/0 0.0.0.0/0
match TTL > 10
0 0 ACCEPT all -- * * 0.0.0.0/0 0.0.0.0/0 state RELATED,ESTABLISHED
SHED
0 0 ACCEPT all -- * lo 0.0.0.0/0 0.0.0.0/0
0 0 ACCEPT tcp -- * * 0.0.0.0/0 0.0.0.0/0 tcp dpt:22
0 0 ACCEPT tcp -- * * 0.0.0.0/0 0.0.0.0/0 tcp dpt:80
0 0 ACCEPT tcp -- * * 0.0.0.0/0 0.0.0.0/0 tcp dpt:443
9 540 DROP all -- * * 0.0.0.0/0 0.0.0.0/0
```

Рисунок 13 – блокировка не разрешенного трафика

3.2 Проверка блокировки больших пакетов с TTL > 10

Попробуем отправить большой пакет с TTL > 10 и обычный

```
ping -s 1000 -t 15 192.168.1.10
```

```
ping -c 3 192.168.1.10
```

-s 1000: размер пакета 1000 байт (> 600)

-t 15: TTL = 15 (> 10)

Пакет должен быть заблокирован

Проверим счетчики правил

```
iptables -L -v -n | grep "length.ttl"
```

Счетчики должны увеличиваться

```
(root@kali)-[/home/kali]
# ping -c 3 192.168.1.10
PING 192.168.1.10 (192.168.1.10) 56(84) bytes of data.
64 bytes from 192.168.1.10: icmp_seq=1 ttl=64 time=0.853 ms
64 bytes from 192.168.1.10: icmp_seq=2 ttl=64 time=1.27 ms
64 bytes from 192.168.1.10: icmp_seq=3 ttl=64 time=0.914 ms

— 192.168.1.10 ping statistics —
3 packets transmitted, 3 received, 0% packet loss, time 2002ms
rtt min/avg/max/mdev = 0.853/1.011/1.267/0.182 ms

(root@kali)-[/home/kali]
# ping -s 1000 -t 15 192.168.1.10
PING 192.168.1.10 (192.168.1.10) 1000(1028) bytes of data.
^C
— 192.168.1.10 ping statistics —
29 packets transmitted, 0 received, 100% packet loss, time 29563ms
```

Рисунок 14 – проверка отправки больших и обычных пакетов

3.3 NAT

Убедимся, что транзитный трафик проходит через NAT

На InternalClient:

Попробуем ping к ExternalClient через шлюз

```
ping -c 3 192.168.1.10
```

На IPtablesFW (мониторинг):

В отдельном терминале проверим NAT

```
sudo tcpdump -i eth0 -n -v
```

Проверим правила NAT

```
iptables -t nat -L POSTROUTING -v -n
```

Должно быть правило MASQUERADE для eth0

Проверим connection tracking

```
cat /proc/net/nf_conntrack | head -5
```

Должны быть записи о NAT-трансляции

- tcpdump: Это инструмент для захвата и анализа сетевого трафика. `-i eth0` указывает интерфейс для мониторинга, `-n` отключает разрешение имен (показывает IP вместо доменных имен), `-v` увеличивает детализацию вывода. В tcpdump можно увидеть реальные IP-адреса в пакетах - если NAT работает, исходный IP будет изменен на IP внешнего интерфейса.

- Connection tracking (`nf\_conntrack`): Это механизм ядра Linux, который отслеживает состояние сетевых соединений. Он необходим для работы NAT - ядро запоминает соответствие между внутренним и внешним IP/портами, чтобы правильно транслировать ответные пакеты обратно. Файл `/proc/net/nf\_conntrack` содержит таблицу отслеживаемых соединений.

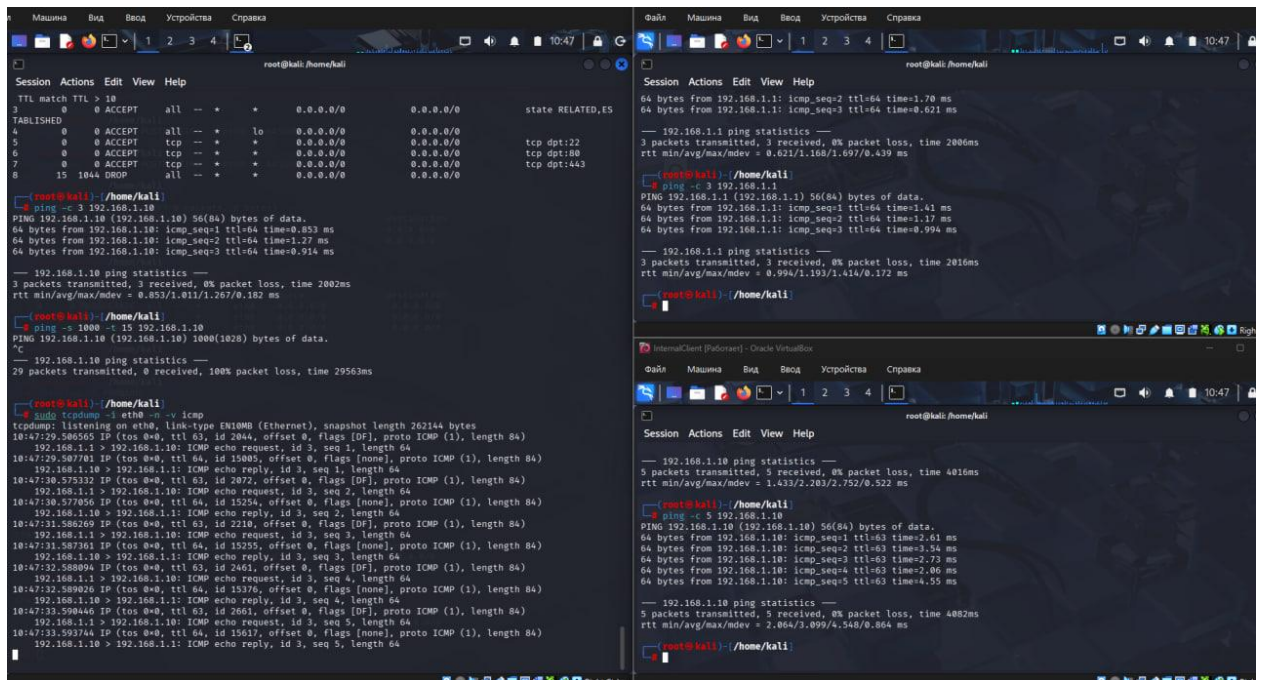


Рисунок 15 - пинг от internalclinet к externalclient

3.4 Защита от brute force

Убедимся, что после 5 попыток SSH подключения адрес блокируется только для SSH
На ExternalClient:

Попробуем подключиться к SSH 6 раз подряд

```
for i in {1..6}; do
  echo "Попытка $i"
  ssh -o ConnectTimeout=2 -o StrictHostKeyChecking=no kali@192.168.1.1
  sleep 1
done
```

После 5 попыток адрес должен быть заблокирован для SSH

6-я попытка не должна пройти

На IPtablesFW:

Проверим заблокированные адреса

```
cat /proc/net/xt_recent/SSH
```

Должен быть IP-адрес ExternalClient (192.168.1.10)

Проверим счетчики цепочки SSH_BRUTEFORCE

```
iptables -L SSH_BRUTEFORCE -v -n
```

Должны быть счетчики для правил DROP

- `/proc/net/xt_recent/SSH`: Это файл в виртуальной файловой системе `proc`, который содержит список IP-адресов, отслеживаемых модулем `'xt_recent'` для списка с именем "SSH". Здесь хранятся IP-адреса, которые пытались подключиться по SSH, с временными метками последнего обращения.

- Счетчики правил (`-v`): Опция `-v` (verbose) показывает счетчики пакетов и байт для каждого правила. Колонка "pkts" показывает количество пакетов, обработанных правилом, "bytes" - количество байт. Это полезно для диагностики - можно увидеть, сколько пакетов было заблокировано или разрешено каждым правилом.

```
(root@kali) ~ # for i in {1..6}; do
echo "Попытка $i"
ssh -o ConnectTimeout=2 -o StrictHostKeyChecking=no kali@192.168.1.1
sleep 1
done
Попытка 1
ssh: connect to host 192.168.1.1 port 22: Connection refused
Попытка 2
ssh: connect to host 192.168.1.1 port 22: Connection refused
Попытка 3
ssh: connect to host 192.168.1.1 port 22: Connection refused
Попытка 4
ssh: connect to host 192.168.1.1 port 22: Connection refused
Попытка 5
ssh: connect to host 192.168.1.1 port 22: Connection timed out
Попытка 6
ssh: connect to host 192.168.1.1 port 22: Connection timed out

(root@kali) ~ # cat /proc/net/xt_recent/SSH
src=192.168.1.1 ttl: 64 last_seen: 4296585684 oldest_pkt: 14 4296583401, 4296583658, 4296583913, 4296584169, 4
296584424, 4296584424, 4296584683, 4296584683, 4296585165, 4296585165, 4296585425, 4296585425, 4296585684, 429
6585684
```

Рисунок 16 – проверка защиты от brute force

Важно отметить что пинг при это все равно работает, блокируется исключительно SSH соединение

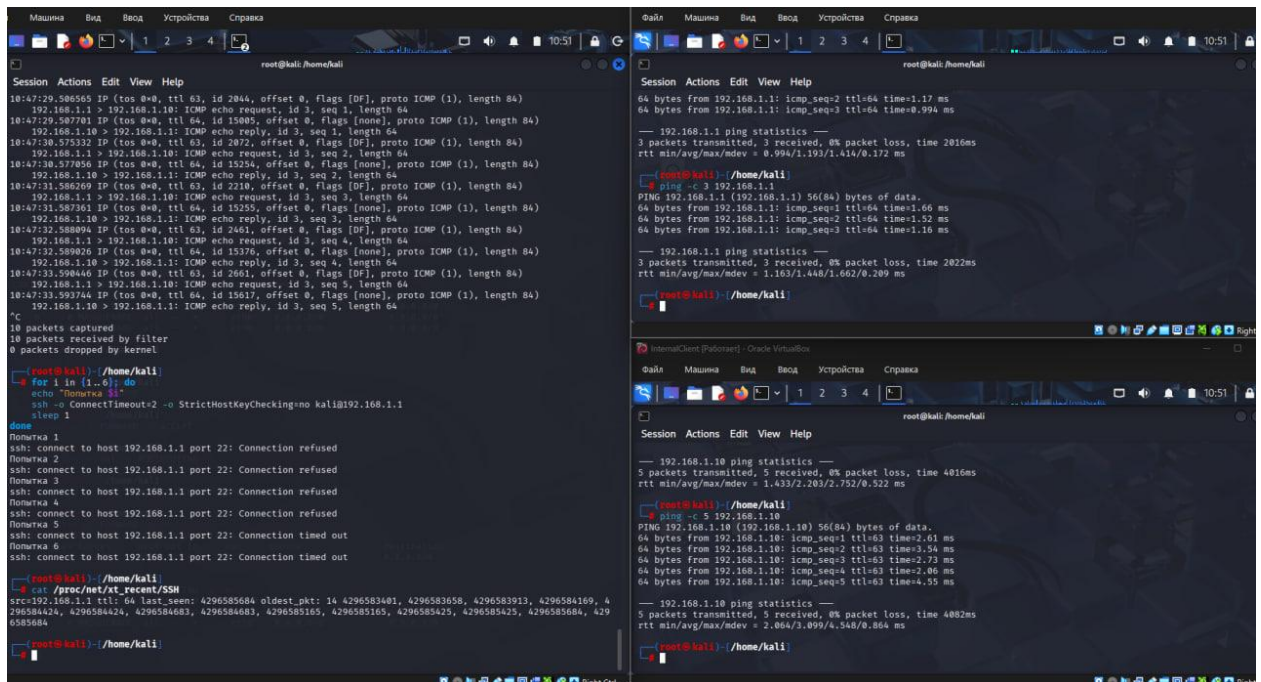


Рисунок 17 – проверка работоспособности команды ping после введения блокировки при brute force

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы была успешно спроектирована и настроена сетевая инфраструктура с применением межсетевого экрана iptables для защиты сетевого уровня (L3) и транспортного уровня (L4).

1. Реализована базовая связность: с помощью технологии Internal Network в VirtualBox созданы две изолированные сети (external-net и internal-net). Настройка IP forwarding на шлюзе IPtablesFW обеспечила маршрутизацию между сегментами, а ручная настройка IP-адресов и маршрутов по умолчанию – корректную адресацию всех узлов сети. Экспериментально подтверждена работоспособность транзитного трафика между сетями;

2. обеспечена фильтрация исходящего трафика: настройка правил в цепочке OUTPUT предотвращает несанкционированные исходящие соединения. Реализована политика "разрешить по умолчанию" для портов SSH (22), HTTP (80) и HTTPS (443), при этом весь остальной исходящий трафик блокируется. Тестирование показало успешную блокировку неразрешенных протоколов (например, ICMP для ping) при сохранении работоспособности разрешенных сервисов;

3. обеспечена защита от подозрительных пакетов: внедрение правил фильтрации по размеру пакета и значению TTL позволило блокировать пакеты размером более 600 байт с TTL больше 10. Правила применены к цепочкам INPUT, OUTPUT и FORWARD, что обеспечивает комплексную защиту на всех этапах обработки пакетов. Экспериментально подтверждена эффективность фильтрации: обычные пакеты (84 байта) проходят нормально, а подозрительные большие пакеты с высоким TTL успешно блокируются;

4. реализована трансляция сетевых адресов: настройка NAT с использованием MASQUERADE в таблице nat обеспечила скрытие внутренней сетевой топологии. Все пакеты от внутренних узлов (10.0.0.0/24) при прохождении через шлюз получают исходный IP внешнего интерфейса (192.168.1.1). Тестирование с использованием tcpdump показало успешную трансляцию адресов и корректную работу connection tracking для обратной трансляции ответных пакетов;

5. защита от атак перебора паролей: настройка пользовательской цепочки SSH_BRUTEFORCE с использованием модуля recent обеспечила защиту SSH-сервера от brute force атак. Реализована логика блокировки IP-адресов после 5 попыток подключения за 10 минут. Тестирование показало успешную блокировку атакующих адресов только для SSH-трафика, при этом другие сервисы (ping, HTTP) продолжают работать нормально, что демонстрирует селективность механизма защиты.

Все поставленные в лабораторной работе задачи выполнены в полном объеме. Результаты тестирования подтверждают, что применение комплексного подхода к безопасности сетевого и транспортного уровней с использованием iptables критически важно для устойчивости современной сетевой инфраструктуры. Приобретенные навыки настройки правил фильтрации пакетов, работы с NAT и защиты от сетевых атак могут быть успешно применены в реальных проектах по обеспечению информационной безопасности.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Олифер В. Г., Олифер Н. А. Компьютерные сети. Принципы, технологии, протоколы : учебник для вузов. – 5-е изд. – СПб. : Питер, 2016. – 992 с.
2. Куроуз Дж. Ф., Росс К. В. Компьютерные сети: Нисходящий подход / пер. с англ. – 6-е изд. – М.: Эксмо, 2016. – 912 с.
3. OpenNET. Руководство по iptables (Iptables Tutorial) : сайт. – Текст : электронный. – URL: <https://www.opennet.ru/docs/RUS/iptables/> (дата обращения: 20.02.2026).
4. Habr. Iptables для начинающих : блог. – Текст : электронный. – URL: <https://habr.com/ru/articles/747616/> (дата обращения: 20.02.2026).
5. StuffPhilWrites. Iptables Processing Flowchart : блог. – Текст : электронный. – URL: <https://stuffphilwrites.com/2014/09/iptables-processing-flowchart/> (дата обращения: 20.02.2026).